IMPLEMENTATION GUIDE / 01

Git Good Firebase & navigation

A detailed walkthrough of the work completed in this chat

Project: Git Good Android app
Package: com.example.gitgood
Prepared: 31 October 2026

What the app does now

The app opens an authentication flow instead of the original Hello Android screen. Users can choose email/password login, create an account, request a password-reset email, or continue with Google. Firebase authentication state determines whether the app displays the welcome flow or the signed-in home destination.

How the pieces connect

```
Compose screens
      | user actions
      v
LoginViewModel  &lt;--- StateFlow ---&gt;  MainActivity
      |                                    |
      v                                    v
AuthRepository                    Navigation 3 back stack
      |                                    |
      v                                    v
Firebase Authentication            NavDisplay / screen entries

Google button -&gt; Credential Manager -&gt; ID token
                                       |
                                       v
                                Firebase credential
```

Nudj supplied the architectural reference: Hilt injection, a repository layer, observable ViewModel state, Credential Manager, and typed Navigation 3 entries in MainActivity. Git Good uses these ideas for its own screens and Firebase project.

Verification boundary: debug builds passed. Existing unit tests passed after the authentication wiring. The final Navigation 3 update also built successfully. A real account login or Google account-picker test on a device was not performed.

IMPLEMENTATION GUIDE / 02

Where everything lives

All paths below are relative to the project root: **/Users/apple/Desktop/gitgood**. Kotlin paths start under **app/src/main/java/com/example/gitgood/**.

File or directory	Responsibility
app/google-services.json	Firebase configuration for com.example.gitgood; latest supplied replacement was google-services (9).json.
app/build.gradle.kts	Applies plugins and includes Firebase, Google identity, Hilt, coroutine and Navigation 3 dependencies.
gradle/libs.versions.toml	Central versions, library aliases and plugin aliases.
build.gradle.kts	Declares shared plugin versions through catalog aliases, with apply false.
app/src/main/AndroidManifest.xml	Internet permission and registered GitGoodApplication class.
GitGoodApplication.kt	Hilt-enabled Application entry point.
di/AuthModule.kt	Provides a singleton FirebaseAuth instance.
data/repository/AuthRepository.kt	Email login, registration, Google token exchange, reset email and auth-state Flow.
data/repository/GoogleSignInClient.kt	Credential Manager request, Google ID-token parsing and credential-state clearing.
viewModel/LoginScreenViewModel.kt	Contains the LoginViewModel class and coordinates auth actions/UI state.
navigation/ScreenRoute.kt	Serializable typed destinations implementing NavKey.
navigation/AppNavigation.kt	Currently contains RegisterDestination and HomeDestination composables. MainActivity owns navigation.
MainActivity.kt	Collects session/UI state and declares Navigation 3 entries and navigation callbacks.

Screen files

The login layout and LoginUiState are under ui/theme/screen/auth/login/. Registration is under ui/theme/screen/auth/SignUp/. Those files declare com.example.gitgood.ui.screen.auth.login and ...auth.register packages, even though their physical directories include ui/theme/screen. Kotlin resolves imports by package declarations.

SignUpScreenViewModel.kt remains an empty class. The implemented LoginViewModel currently coordinates registration as well as login. No separate active AuthViewModel is used in the final structure.

IMPLEMENTATION GUIDE / 03

Firebase configuration & certificates

Why the JSON belongs inside app/

The Android app module applies the Google Services plugin. That plugin reads app/google-services.json, selects the client matching the module applicationId, and generates Android resources used by Firebase and Google identity. Placing the file only at the repository root does not provide the expected module configuration.

```
app/
  google-services.json
  build.gradle.kts
  src/
gradle/
build.gradle.kts
settings.gradle.kts
```

How the package mismatch was resolved

The earlier supplied files registered com.example.circuitly and then com.company.gitgood. The app itself used com.example.gitgood. Google Services processing correctly rejected those mismatches. You chose to retain com.example.gitgood and supplied a matching file. Later replacements overwrote the same app/google-services.json rather than adding numbered config files to the module.

The latest inspected JSON contains Android and web OAuth client entries (client types 1 and 3). The generated default_web_client_id resource comes from the web client configuration. The helper uses that resource rather than a hard-coded client ID.

Debug certificate fingerprints generated

The signingReport task read the debug certificate from /Users/apple/.android/debug.keystore, alias AndroidDebugKey. These are certificate fingerprints, not hashes of the JSON file.

```
SHA-1
A6:B5:89:2C:AC:09:B5:9A:5E:DE:9A:BB:CF:38:9D:74:92:4A:E4:A7

SHA-256 (continued on next line)
01:C8:63:A9:41:3B:26:76:0D:57:7D:78:5C:EF:DF:36:
B9:62:E9:CB:C9:7A:DF:46:2D:1B:EE:53:C5:0F:8D:98
```

For SHA-256, join both lines into one value when pasting. These fingerprints apply to this debug key. Release signing was not configured in the signing report; a production signing certificate will have its own fingerprints.

> Firebase console actions were not performed by the agent. Confirm that the correct Android app has these fingerprints and that Email/Password and Google providers are enabled. Download a refreshed JSON if the console changes its OAuth configuration.

IMPLEMENTATION GUIDE / 04

Gradle plugins & dependencies

The version catalog centralizes shared versions. The root build declares plugin aliases with apply false; the app module applies the plugins it needs. A malformed [libraries] header was also corrected during setup.

Component	Configured version / purpose
Google Services	4.5.0 - turns matching Firebase JSON into app resources.
Firebase BoM	34.11.0 - selects mutually compatible Firebase library versions.
Firebase Authentication	com.google.firebase:firebase-auth - no explicit version because the BoM supplies it.
Credential Manager	1.3.0 - credentials and credentials-play-services-auth.
Google ID library	1.1.1 - Google request options and ID-token credential parsing.
Hilt	2.60.1 - Android runtime, compiler and Gradle plugin.
KSP	2.3.9 - runs Hilt symbol processing for Kotlin sources.
Navigation 3	1.1.1 - runtime and UI libraries.
Lifecycle integration	2.11.0 - ViewModel, lifecycle-aware collection and Navigation 3 ViewModel decorator.
Kotlin serialization	Plugin 2.2.10; core library 1.9.0 - serializes typed navigation keys.
Coroutine Task adapter	kotlinx-coroutines-play-services 1.10.2 - suspends while Firebase Tasks complete.

Why the Firebase version changed

The initially added BoM 35.1.0 selected Firebase Auth 25.0.0. The build reported Kotlin metadata 2.4.0 while this project expected 2.2.0. The BoM was changed to 34.11.0, after which compilation and subsequent debug builds passed. This avoided changing the app compiler just to satisfy that dependency.

What remains in the catalog

The Navigation 2 navigation-compose alias and its 2.9.8 version remain defined, but the final app dependency list uses Navigation 3. A catalog alias by itself does not add a library to the APK.

IMPLEMENTATION GUIDE / 05

Hilt: who constructs each object?

Dependency injection means the app asks for its collaborators through constructors, while Hilt generates the code that builds and connects them. This keeps Firebase setup out of the composable layouts.

```
@HiltAndroidApp
class GitGoodApplication : Application()

@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    private val loginViewModel: LoginViewModel by viewModels()
}
```

The manifest registers .GitGoodApplication using android:name. @HiltAndroidApp creates the application-level dependency graph. @AndroidEntryPoint lets MainActivity participate in that graph. The activity ViewModel delegate can then obtain the Hilt-created LoginViewModel.

FirebaseAuth comes from a module

```
@Module
@InstallIn(SingletonComponent::class)
object AuthModule {
    @Provides @Singleton
    fun provideFirebaseAuth(): FirebaseAuth =
        FirebaseAuth.getInstance()
}
```

FirebaseAuth is an SDK class, so our code supplies it through @Provides instead of annotating its constructor. SingletonComponent gives the binding application lifetime; @Singleton lets consumers share the same provided instance.

The construction chain

```
GitGoodApplication / Hilt graph
  -&gt; FirebaseAuth from AuthModule
  -&gt; AuthRepository(FirebaseAuth)
  -&gt; GoogleSignInClient()
  -&gt; LoginViewModel(AuthRepository, GoogleSignInClient)
  -&gt; MainActivity observes the ViewModel
```

AuthRepository has @Inject constructor and @Singleton. GoogleSignInClient also has an injectable constructor. LoginViewModel has @HiltViewModel and an @Inject constructor. The Hilt compiler runs through ksp(libs.hilt.compiler); the Gradle plugin supplies the Android integration and generated-code transformations.

The ViewModel is scoped to MainActivity, so login and registration share one auth operation state. Navigation 3 also installs a ViewModel-store decorator for entries, but the current LoginViewModel is intentionally obtained from the activity, not separately for each screen.

IMPLEMENTATION GUIDE / 06

AuthRepository: the Firebase boundary

AuthRepository is the small layer responsible for SDK calls. It accepts credentials, performs Firebase operations, and exposes the current authentication session. It does not draw screens or decide which route should appear.

Member	Behavior
currentUser	Returns FirebaseAuth.currentUser at the time it is read.
user	A callbackFlow of FirebaseUser? from Firebase auth-state changes.
signIn(email, password)	Trims the email and awaits signInWithEmailAndPassword.
signUp(email, password)	Trims the email and awaits createUserWithEmailAndPassword.
signInWithGoogle(token)	Builds a Google Firebase credential and awaits signInWithCredential.
resetPassword(email)	Awaits Firebase sendPasswordResetEmail.
signOut()	Calls FirebaseAuth.signOut().

From callbacks to Flow

```
val user = callbackFlow {
    val listener = FirebaseAuth.AuthStateListener {
        trySend(it.currentUser)
    }
    auth.addAuthStateListener(listener)
    awaitClose { auth.removeAuthStateListener(listener) }
}
```

Firebase reports a user for a signed-in session and null for a signed-out session. callbackFlow adapts the listener into a coroutine Flow. awaitClose removes the listener when collection stops, so a cancelled collector does not leave its listener attached.

Why await() matters

Firebase methods return Task objects. The coroutine adapter await() suspends until a Task completes instead of blocking the UI thread. A failed Task becomes an exception, which travels back to the ViewModel. The ViewModel then displays the error and turns off its loading state.

The repository does not save passwords to a database, write a user profile document, or implement app-specific authorization. Firebase handles the authentication session. This integration does not add Firestore, Realtime Database, role checks, or user-profile onboarding.

IMPLEMENTATION GUIDE / 07

LoginViewModel: state & actions

The UI state

LoginUiState contains email, password, isPasswordVisible, isLoading, errorMessage and message. The ViewModel owns a MutableStateFlow privately and exposes a read-only StateFlow. Composables receive state and callbacks rather than calling Firebase directly.

Input and validation

onEmailChange / onPasswordChange: update input and clear the displayed error. **togglePasswordVisibility:** toggles isPasswordVisible; this fixes the earlier unfinished ViewModel reference to a non-existent passwordVisible field. **clearMessage:** clears both error and informational messages.

onLoginClick: validates the trimmed email with Android Patterns.EMAIL_ADDRESS, rejects a blank password, then asks the repository to sign in. **onSignUpClick:** checks email format, a minimum six-character password and matching confirmation before creating the account. Firebase may still enforce stricter console-side rules.

One shared operation wrapper

```
runAuth {
    // Perform a suspend authentication operation.
}

// runAuth sequence:
// 1. Ignore another submission while loading.
// 2. Set loading; clear previous messages.
// 3. Launch work in viewModelScope.
// 4. Clear login password after success.
// 5. Handle cancellation or expose an error.
// 6. Always turn loading off in finally.
```

Dismissing the Google account picker is treated as a normal user cancellation. Coroutine CancellationException is rethrown so lifecycle cancellation works correctly. Other exceptions become a displayed error using the exception message or a fallback. finally resets loading in all cases.

Session state is separate from form state

currentUser converts the repository user Flow into StateFlow with stateIn(viewModelScope, SharingStarted.Eagerly, authRepository.currentUser). It starts with the SDK current user and listens for changes while the ViewModel lives. MainActivity observes this session state to navigate after sign-in and sign-out.

> The ViewModel survives activity recreation. It is not durable storage. Registration passwords use remember rather than rememberSaveable; they are not intentionally written into saved navigation state. Login input can remain in the activity ViewModel until updated or cleared.

IMPLEMENTATION GUIDE / 08

Google sign-in, step by step

Google identity and Firebase Authentication are two connected steps. The account picker obtains a Google ID token; Firebase exchanges that token for a Firebase-authenticated session.

Step	What happens in this app
1. User taps Google	LoginScreenLayout invokes onGoogleClick. MainActivity passes the current UI context to LoginViewModel.
2. Build request	GoogleSignInClient builds GetSignInWithGoogleOption using R.string.default_web_client_id.
3. Show account UI	CredentialManager.getCredential receives the request and UI context. The helper does not keep a Context field.
4. Validate credential	The response must be CustomCredential of the Google ID-token credential type.
5. Read ID token	GoogleIdTokenCredential.createFrom parses the credential data and returns idToken.
6. Authenticate Firebase	AuthRepository creates GoogleAuthProvider.getCredential(token, null) and calls signInWithCredential.
7. Observe new user	The Firebase auth listener emits the user. MainActivity resets the stack to Home.

Why the web client ID is used

The request uses the server/web OAuth client ID generated into default_web_client_id, not the Android client ID. The Google Services plugin supplies this resource from the matching JSON configuration. The Kotlin helper therefore does not embed an OAuth client string.

Cancellation, failures and sign-out

A cancelled picker does not show a failure message. An unexpected credential type or SDK exception reaches the ViewModel error path. On sign-out, the app first signs out of Firebase, then calls clearCredentialState so Credential Manager can clear the app credential-session state.

The Google button appears in login and registration. Firebase Google sign-in can create a Firebase account for a new Google identity. Git Good does not copy Nudj's restrictions that require an existing user profile, a particular institutional email domain, or a selected role.

> No real Google account credentials were entered by the agent. Successful compilation confirms API usage and resource generation, but it does not prove the console provider, account picker, signing certificate and device environment all work together.

IMPLEMENTATION GUIDE / 09

Navigation in MainActivity

The final navigation follows Nudj's MainActivity pattern: typed route objects, rememberNavBackStack, NavDisplay, an entryProvider, and a session-driven LaunchedEffect. The previous Boolean register flag is no longer the navigation mechanism.

```
@Serializable
sealed interface ScreenRoute : NavKey {
    @Serializable
    sealed interface Auth : ScreenRoute {
        @Serializable data object Splash : Auth
        @Serializable data object Landing : Auth
        @Serializable data object Login : Auth
        @Serializable data object Register : Auth
    }
    @Serializable data object Home : ScreenRoute
}
```

The stack is the navigation state

rememberNavBackStack starts with Splash and can save/restore serializable route keys. NavDisplay renders entries for those keys. The saveable-state decorator preserves destination saveable state; the ViewModel-store decorator supports entry-scoped ViewModel lifetimes. MainActivity also collects auth and form StateFlows with collectAsStateWithLifecycle.

Event	Stack behavior
Initial signed-out state	Splash is replaced by Landing. No artificial splash delay is added.
Welcome: Log in	Login is pushed after Landing.
Welcome: Get started	Register is pushed after Landing.
Switch Login / Register	Current auth form entry is replaced, preventing repeated form-screen stacking.
System Back	If more than one entry exists, the top is removed and messages are cleared.
User becomes signed in	Stack is cleared; Home becomes its only entry.
User signs out	A stack containing Home is cleared and replaced with Landing.

The auth effect is keyed to user?.uid. Ordinary form recomposition does not reset the stack. For an unauthenticated user already on a restored Login or Register route, it leaves that route intact. It redirects only from Splash or a stack that still contains Home.

IMPLEMENTATION GUIDE / 10

Screen wiring & user journeys

Login layout

LoginScreenLayout remains a stateless UI function: it receives LoginUiState and callbacks. It displays validation/backend errors and password-reset messages, provides a password-visibility toggle, and disables the login and Google buttons during loading. A Google logo and Continue with Google action were added. The login copy now says Git Good.

Registration destination

RegisterDestination wraps the existing RegisterScreenLayout. It keeps email in rememberSaveable, passwords and visibility in remember, and passes loading/error state from LoginViewModel. It invokes onSignUpClick with email, password and confirmation, or invokes the same Google flow as login. The existing empty SignUpScreenViewModel is not used by this destination.

Password reset

The Forgot password action uses the email currently entered on the login form. A missing or malformed email produces a validation error. A valid email starts sendPasswordResetEmail; a completed request displays: If an account exists, a password reset email will be sent. The app does not claim that an address is registered.

There is no custom in-app reset-code page or incoming reset-password deep-link handler. The integration sends Firebase's reset email. Nudj's custom reset/verification navigation was not transferred.

Signed-in home

HomeDestination is a simple confirmation screen displaying Welcome to Git Good, the current email when available, and a Sign out button. It is an authentication destination placeholder, not a finished Git Good dashboard.

Example: email login

```
Landing -&gt; Login -&gt; enter email/password -&gt; Log in
  -&gt; validate -&gt; isLoading = true -&gt; Firebase request
  -&gt; Firebase user emits -&gt; stack becomes [Home]

Wrong password or network error:
  -&gt; errorMessage shown -&gt; isLoading = false
  -&gt; user can correct the input and retry
```

Example: account creation

Landing -> Get started -> Register -> enter matching credentials -> Sign up. Successful Firebase account creation signs the user in, so the session observer sends the app to Home. Email verification is not currently required or implemented.

IMPLEMENTATION GUIDE / 11

What was checked, and what remains

Checks completed during this work

Check	Observed result and scope
JSON client matching	The final Firebase file matched com.example.gitgood. Earlier mismatches were detected by Google Services processing.
processDebugGoogleServices	Passed after matching configuration replacements. Confirms JSON processing and generated resources.
signingReport	Produced the debug SHA-1 and SHA-256 values. Release signing had no configuration.
compileDebugKotlin	Passed for auth, Google identity, Hilt and navigation code.
KSP / Hilt tasks	Passed, including generated dependency code and Android integration during assembleDebug.
assembleDebug	Passed after Firebase integration and after the final Navigation 3 migration. Latest observed build: BUILD SUCCESSFUL in 2m 27s.
testDebugUnitTest	Passed after the authentication wiring. These were existing unit tests, not a new end-to-end auth test suite.
git diff --check	Passed for whitespace checks during implementation.

Not verified or not implemented

No real-device or emulator account sign-in was performed. Provider configuration in Firebase Console was not inspected or changed. Release signing, Play App Signing fingerprints, custom verification/reset deep links, email-verification gating, profile storage, role authorization and a complete signed-in dashboard are not part of the completed work.

Intentional differences from Nudj

Git Good reuses the architecture and navigation pattern, but not Nudj's product rules. There is no institutional-domain restriction, role selector, club approval flow or Firestore user-existence check. It has one activity-scoped LoginViewModel for the current auth flow rather than all of Nudj's separate destination ViewModels.

> The main remaining validation step is to install the debug app and exercise email login, registration, Google sign-in, reset email, rotation, Back, app restart and sign-out. A successful build is necessary but is not evidence that these device-level journeys have been executed.

IMPLEMENTATION GUIDE / 12

Runbook, troubleshooting & references

Useful local commands

```
./gradlew :app:processDebugGoogleServices
./gradlew :app:signingReport
./gradlew :app:compileDebugKotlin
./gradlew :app:assembleDebug
./gradlew :app:testDebugUnitTest
```

Debug APK: app/build/outputs/apk/debug/app-debug.apk. Run it through Android Studio to test the interactive journeys.

If something goes wrong

Symptom	What to check first
No matching client found	Compare applicationId with the JSON Android package. Download the file for com.example.gitgood.
Missing default_web_client_id	Confirm Google is enabled, download updated JSON with a web OAuth client, and run Google Services processing again.
Google picker/sign-in fails	Check provider enablement, the debug certificate in Firebase, matching OAuth configuration and device Google services. Read the actual SDK error.
Kotlin metadata incompatibility	Compare the dependency compiler requirements with this project. BoM 34.11.0 was the verified workaround for the earlier Firebase Auth 25.0.0 mismatch.
Hilt setup error	Check @HiltAndroidApp, manifest android:name, @AndroidEntryPoint, @HiltViewModel and KSP/plugin setup.
Reset email not received	Check provider setup, the entered address, mailbox/spam, network and Firebase error output.

Key terms

Repository: SDK/data access layer. **ViewModel:** screen logic and observable state. **StateFlow:** stream holding a current value. **Coroutine:** cancellable asynchronous work. **Hilt:** generated dependency construction. **ID token:** Google identity proof exchanged with Firebase. **Back stack:** ordered route keys describing navigation history.

Source of truth and further reading

Source: current Git Good project files and recorded build results. Nudj was read as a reference and was not modified.

Firebase Android setup

Firebase Google sign-in

Hilt Gradle setup

Navigation 3 basics